



Ecole Nationale Supérieure d'Informatique et d'Analyse des Systèmes - RABAT

Département Web and Mobile Engineering

Filière : Business intelligence & Analytics

Projet de fin de la 1ère année

An LLM based recipe generator and meal planner App.

Présenté par :

Mohamed Oussama Rafiq

Mohamed Amine Tarik

Sous la supervision de :

Prof H.Benbrahim

Année Académique : 2024-2025

Résumé

Ce rapport présente la conception et la réalisation d'une application web innovante de génération et planification de recettes culinaires, basée sur un système Retrieval-Augmented Generation (RAG) associé à un grand modèle de langage (LLM), LLaMA 4. Face aux limites des solutions existantes, notamment le manque de ressources spécifiques à la cuisine marocaine et les hallucinations des modèles généralistes comme ChatGPT, notre projet vise à valoriser et diffuser cette richesse culinaire locale à travers une solution locale performante et personnalisable. Le travail inclut la collecte, le prétraitement et l'intégration de datasets internationaux et marocains, l'adaptation des embeddings pour mieux gérer les termes en darija et les particularités culturelles, ainsi que le développement d'une interface web sécurisée et responsive avec Flask et SQLite. Les défis techniques rencontrés, tels que la compatibilité des données et la reconnaissance linguistique, ont été surmontés par des solutions hybrides et des reformulations intelligentes des requêtes. Les résultats montrent que le système propose des réponses pertinentes, personnalisées et fiables, tout en offrant une meilleure expérience utilisateur par rapport aux alternatives actuelles. Ce projet illustre l'importance d'un développement localisé en intelligence artificielle pour valoriser des patrimoines culturels spécifiques.

Table des matières

Table des matières **ii**

Introduction **3**
Contexte . . . 3
Introduction du sujet . . . 3
Méthodologie de travail . . . 4
Étapes de mise en œuvre . . . 4
Cahier de charge . . . 5
Principales Fonctionnalités . . . 5
Composants Fonctionnels . . . 6
Étapes de Mise en Œuvre . . . 7
Roadmap . . . 7

Chapitre 1 : Les Modèles de Langage et l'Approche RAG : **8**
Introduction . . . 8
1.1. Les Modèles de Langage (LLM) . . . 8
1.2. L'Approche Retrieval-Augmented Generation (RAG) . . . 9
1.3. Embeddings . . . 10
Conclusion . . . 11

Chapitre 2 : Conception de la solution **12**
Introduction . . . 12
Partie 1 : Étude des solutions existantes . . . 12
1.1 Applications existantes et systèmes similaires . . . 12
1.2 Limites spécifiques observées . . . 13
Partie 2 : Conception de notre solution . . . 14
2.1 Un besoin local et culturel . . . 14
2.2 Présentation de la solution proposée . . . 14
2.3 Architecture globale du système . . . 14
2.4 Sources de données et nutrition . . . 15
Tableau comparatif des solutions . . . 16
Conclusion du chapitre . . . 17

Chapitre 3 : Réalisation **18**
3.1 Vue d'ensemble de l'architecture . . . 18
3.2 Pipeline de traitement et système RAG . . . 19
3.2.1 Structure du projet . . . 19

a) Préparation des données . . . 19
b) Génération des embeddings . . . 20
c) Indexation et recherche . . . 20
d) Modèle de génération . . . 20
3.3 Difficultés rencontrées . . . 21
3.4 Interface utilisateur . . . 21
Backend (Flask + SQLite) . . . 21
Frontend (HTML/CSS/JS) . . . 22
Structure de la partie UI . . . 23
Exemple de route d'intégration . . . 23
Conclusion du chapitre . . . 23

Chapitre 4 : Résultats et Interface Utilisateur **25**
4.1 Présentation Générale de l'Interface . . . 25
4.2 Page de Connexion . . . 25
4.3 Page de Chat IA . . . 26
4.4 Exemple de Test . . . 27
4.5 Historique des Conversations . . . 27
4.6 Profil Utilisateur . . . 28

Conclusion Générale **29**

Bibliographie **30**

Introduction

Contexte

La gestion des repas quotidiens reste un défi pour de nombreuses personnes, surtout pour de jeunes étudiants dont la cuisine est devenue leur plus grande préoccupation. C'est une activité qui prend un temps important dans notre vie quotidienne, surtout en essayant de trouver quoi faire avec les ingrédients présents chez nous. Naturellement, beaucoup d'entre nous se tournent vers les blogs culinaires ou les sites de recettes, pour finalement se retrouver submergés par une avalanche de choix. Cependant, le défi ne réside pas seulement dans la variété, mais aussi dans la difficulté de préparation de ces recettes, puisque généralement ce sont des recettes qui exigent une certaine compétence culinaire ou des ingrédients rares, absents de nos placards, ou même impossibles à trouver au supermarché. De plus, les portions suggérées ne correspondent souvent pas aux besoins de toutes les personnes ou les familles. Sans oublier que la majorité des recettes sont adaptées à des cuisines européennes et disposent d'un nombre limité de plats marocains traditionnels comme le tajine, le couscous ou les harchas, dont les recettes en ligne sont inadaptées ou manquent de détails ou d'explications.

Introduction du sujet

Aujourd'hui, avec l'avancée de l'intelligence artificielle qui a proposé de nombreuses solutions dans différents domaines, il devient primordial d'exploiter cette technologie pour surmonter ces défis et aider l'utilisateur à trouver des recettes qui répondent à ses besoins, que ce soit un manque d'ingrédients ou de budget, ou même l'inverse, en incluant des

ingrédients à des recettes pour éviter le gaspillage des aliments, et lui offrir la possibilité de planifier ses repas d'une manière optimale. C'est dans ce cadre que s'inscrit notre projet de PFA de première année : un générateur de recettes et un planificateur de repas basé sur un LLM. Ce projet vise à offrir une interface intuitive où les utilisateurs pourront : Entrer les ingrédients dont ils disposent pour générer des recettes adaptées. Recevoir des suggestions de plats marocains et internationaux tenant compte de leurs contraintes et préférences alimentaires. Obtenir un plan de repas personnalisé sur plusieurs jours, optimisant ainsi la gestion des ressources et du temps. Grâce à cette solution, nous souhaitons proposer un outil pratique et intelligent, facilitant la cuisine au quotidien tout en valorisant le patrimoine culinaire marocain.

Méthodologie de travail

Les Large Language Models (LLM) sont une technologie largement utilisée dans plusieurs domaines. C'est pourquoi on les emploie pour générer des recettes et planifier les repas. Pour adapter notre LLM à notre projet, nous avons le choix entre le fine-tuning et le RAG. On utilisant le Retrieval Augmented Generation (RAG), le modèle peut accéder en temps réel a la base de donnée structurée qui contient les recettes et informations nutritionnelles, garantissant ainsi une réponse pertinente.

Étapes de mise en œuvre

— Collecte et structuration des données contenant une large quantité de recette et d'ingrédient.

— Intégration d'informations nutritionnelles et de suggestions d'alternatives d'ingrédients pour répondre aux différentes contraintes alimentaires.

— Implémentation du RAG : Développement d'un système permettant au LLM d'interacter dynamiquement la base de données pour générer des recettes adaptées aux

ingrédients disponibles.

— Développement de l'interface utilisateur

Création d'un Site Web où l'utilisateur pourra renseigner ses ingrédients disponibles, ses préférences alimentaires et ses objectifs nutritionnels. Finalement la génération des recettes et de plans de repas personnalisés, accompagnés d'instructions claires adaptées aux renseignements demandés par l'utilisateur, et ses préférences.

Cahier de charge

Générateur de Recettes et Planificateur de Repas

Le générateur de recettes et planificateur de repas vise à aider les utilisateurs à optimiser l'utilisation de leurs ingrédients disponibles en générant des recettes uniques et en créant des plans de repas adaptés aux restrictions alimentaires et aux objectifs nutritionnels. L'intégration d'un LLM permet une interface conversationnelle où les utilisateurs peuvent saisir leurs préférences et recevoir des suggestions personnalisées.

Principales Fonctionnalités

Génération de Recettes Basée sur les Ingrédients

— **Saisie des Ingrédients :** Les utilisateurs peuvent entrer les ingrédients dont ils disposent via une interface simple, avec la possibilité de spécifier les quantités.

— **Suggestions de Recettes :** Le LLM génère des recettes en fonction des ingrédients fournis, proposant une variété de plats (entrées, plats principaux, desserts) selon la quantité et le type d'ingrédients disponibles.

Planification Personnalisée des Repas

— **Préférences Alimentaires :** Permet aux utilisateurs d'indiquer leurs restrictions alimentaires (végétarien, végan, sans gluten) et leurs objectifs nutritionnels (riche en protéines, faible en glucides).

— **Plans de Repas Hebdomadaires :** Le LLM propose des plans de repas pour la semaine, incluant des idées de petit-déjeuner, déjeuner, dîner et collations en accord avec les préférences et besoins nutritionnels de l'utilisateur.

Instructions et Conseils de Cuisine

— **Instructions Étape par Étape :** Fournit des instructions claires et concises pour chaque recette, accompagnées de conseils sur la préparation et les techniques de cuisson.

Composants Fonctionnels

Interface Utilisateur (UI)

— **Champ de Saisie des Ingrédients :** Une interface simple et intuitive permettant aux utilisateurs d'entrer leurs ingrédients.

— **Tableau de Bord de Planification des Repas :** Un tableau de bord où les utilisateurs peuvent consulter leurs plans de repas et recettes.

Traitement du Langage Naturel

— **Modèle de Langage (LLM) :** Le moteur principal qui interprète les entrées des utilisateurs, génère des recettes et propose des plans de repas en fonction des données fournies.

Base de Données de Recettes

— **Référentiel de Recettes :** Une base de données structurée contenant des milliers de recettes classées par type, préférences alimentaires et informations nutritionnelles. Ces données peuvent être collectées à partir de diverses sources culinaires.

Étapes de Mise en Œuvre

— Collecte de Données : Réunir un ensemble varié de recettes, d'informations nutritionnelles et de recommandations alimentaires pour entraîner le LLM et construire la base de données de recettes.

— Entraînement et Ajustement du LLM : Affiner le modèle sur des ensembles de données culinaires afin d'améliorer sa compréhension des requêtes liées aux recettes et sa capacité à générer des instructions de cuisine cohérentes.

Roadmap

Notre projet se décompose en trois chapitres fondamentaux :

— Chapitre 1 : Les Modèles de Langage et l'Approche RAG

— Chapitre 2 : Conception de la Solution

— Chapitre 3 : Réalisation et Mise en Œuvre

Chapitre 1 : Les Modèles de Langage et l'Approche RAG :

Introduction

L'intelligence artificielle a révolutionné la manière dont nous interagissons avec l'information, notamment grâce aux modèles de langage de grande taille (LLM - Large Language Models). Ces modèles, entraînés sur d'énormes quantités de données textuelles, sont capables de comprendre et de générer du texte de manière fluide et cohérente. Cependant, malgré leurs performances impressionnantes, ils présentent certaines limites, notamment en ce qui concerne l'accès à des informations spécifiques et actualisées. C'est dans ce contexte que l'approche Retrieval-Augmented Generation (RAG) a été développée. Elle permet de combiner la puissance des LLM avec des bases de données externes, offrant ainsi des réponses plus précises et adaptées aux besoins des utilisateurs. Dans ce chapitre, nous explorerons d'abord le fonctionnement des modèles de langage avant d'expliquer comment le RAG permet de surmonter leurs limitations.

1.1. Les Modèles de Langage (LLM)

Les modèles de langage sont des algorithmes d'intelligence artificielle basés sur des architectures neuronales avancées, comme les Transformers, qui permettent d'analyser et de générer du texte. Entraînés sur d'énormes corpus de données, ils utilisent des techniques statistiques pour prédire le mot suivant dans une phrase, rendant ainsi leurs réponses cohérentes et pertinentes. Leur capacité à traiter et à comprendre le langage naturel en

fait des outils puissants pour de nombreuses applications.

Les LLM offrent plusieurs avantages significatifs. Tout d'abord, ils permettent une génération de texte fluide et naturelle, ce qui les rend adaptés à une interaction conversationnelle. De plus, ils sont capables de comprendre le langage humain et d'interagir de manière contextuelle, ce qui améliore leur pertinence dans les échanges. Enfin, leur polyvalence leur permet d'être utilisés dans divers domaines tels que la traduction, la synthèse d'informations, ou encore la génération de code, élargissant ainsi leur champ d'application.

Cependant, malgré leur efficacité, les LLM présentent certaines limites. Leur mémoire est statique, ce qui signifie qu'ils ne disposent pas d'un accès en temps réel à des bases de données externes pour actualiser leurs connaissances. Ils peuvent également souffrir du phénomène d'hallucination, générant parfois des réponses incorrectes ou inventant des informations. De plus, leur dépendance aux données d'entraînement restreint leur savoir à ce qui a été inclus dans leur phase d'apprentissage, les rendant incapables de traiter des événements récents ou des connaissances spécifiques non couvertes lors de leur entraînement.

1.2. L'Approche Retrieval-Augmented Generation (RAG)

L'approche Retrieval-Augmented Generation (RAG) combine les modèles de langage avec un mécanisme de récupération d'informations provenant de bases de données externes. Plutôt que de s'appuyer uniquement sur son apprentissage préalable, un modèle utilisant le RAG va rechercher des données spécifiques dans une base documentaire avant de générer une réponse. Cette approche permet d'améliorer la pertinence et la précision des réponses fournies par le modèle en l'enrichissant avec des informations actualisées.

Le fonctionnement du RAG repose généralement sur trois étapes principales. Tout d'abord, l'utilisateur soumet une requête, qui peut être une question ou une entrée d'informations spécifiques, comme une liste d'ingrédients disponibles. Ensuite, le système

interroge une base de données externe, telle qu'une base de recettes, afin de récupérer les informations les plus pertinentes en fonction de la requête de l'utilisateur. Enfin, le modèle utilise ces données pour formuler une réponse cohérente et adaptée, en tenant compte des éléments trouvés dans la base documentaire.

L'approche RAG présente plusieurs avantages notables. Elle permet un accès à des informations actualisées et vérifiables, réduisant ainsi le risque d'erreurs et d'hallucinations souvent observé dans les modèles de langage traditionnels. De plus, elle offre une grande flexibilité et une adaptabilité à divers domaines d'application, rendant son utilisation pertinente dans des contextes nécessitant des mises à jour constantes des connaissances, comme la recherche scientifique, la médecine ou encore la génération de recettes de cuisine personnalisées.

1.3. Embeddings

Les embeddings (ou représentations vectorielles) sont des représentations numériques d'objets comme du texte, des images ou du son, conçues pour être traitées par des modèles d'apprentissage automatique. Ils consistent en la transformation de ces objets en vecteurs dans un espace mathématique, en tenant compte de leurs caractéristiques sémantiques et de leur contexte. Ces représentations permettent aux modèles de machine learning d'effectuer des comparaisons de similarité. Par exemple, un modèle peut retrouver des documents, images ou phrases similaires à un élément donné en se basant uniquement sur la proximité entre leurs embeddings respectifs. Cela rend possible des recherches sémantiques avancées, dépassant la simple correspondance de mots-clés. Dans le cadre des grands modèles de langage (LLM), les embeddings jouent un rôle fondamental. Chaque mot, mais aussi son contexte d'apparition dans une phrase ou un paragraphe, est encodé sous forme d'un vecteur. Ce traitement contextuel permet aux modèles d'encoder non seulement le sens isolé des mots, mais aussi leur signification relative à leur environnement linguistique. Par conséquent, les LLM sont capables de capturer des nuances de sens

dans des textes complexes, ce qui leur permet de répondre à des requêtes sémantiques de haut niveau. En utilisant des bases de données vectorielles (comme FAISS), il devient alors possible de stocker ces embeddings et d'effectuer des recherches efficaces et précises. L'usage des embeddings contextuels permet également de mémoriser certaines interactions ou documents dans un format condensé, facilitant leur réutilisation et optimisant les performances des modèles. Ainsi, les embeddings représentent un pilier fondamental de nombreuses applications d'IA modernes, y compris les systèmes de recommandation, les moteurs de recherche intelligents, les chatbots et, bien entendu, les systèmes RAG (Retrieval-Augmented Generation).

Conclusion

Les modèles de langage sont des outils puissants, mais ils présentent des limites lorsqu'il s'agit de générer des informations spécifiques et actualisées. L'approche RAG permet de pallier ces faiblesses en combinant la génération de texte avec une récupération intelligente des données. Cette technologie joue un rôle clé dans notre projet, où elle sera utilisée pour fournir des recettes adaptées aux ingrédients et préférences des utilisateurs, garantissant ainsi des suggestions réalistes et pertinentes.

Chapitre 2 : Conception de la solution

Introduction

Dans ce chapitre, nous présentons la conception de notre générateur intelligent de recettes et planificateur de repas basé sur l'intelligence artificielle. L'analyse des solutions existantes nous permet de mieux cerner leurs limites, notamment en ce qui concerne la personnalisation, la prise en charge de la cuisine marocaine, et la qualité des réponses générées. Notre solution propose une approche innovante, locale et culturellement enrichie, qui s'appuie sur une architecture RAG et un LLM de dernière génération (LLaMA 4 via l'API Groq).

Partie 1 : Étude des solutions existantes

1.1 Applications existantes et systèmes similaires

a. Supercook

Supercook permet de proposer des recettes à partir d'une liste d'ingrédients disponibles chez l'utilisateur. Cependant, il présente certaines limitations importantes :

— Aucune possibilité de choisir un plat spécifique.

— Personnalisation très limitée : pas de prise en compte des préférences culturelles ou diététiques détaillées.

— L'algorithme repose sur une logique simple d'appariement, sans réelle compréhension sémantique.

b. Yummly

Yummly est une plateforme de suggestion de recettes avec filtres diététiques. Toutefois, elle reste centrée sur la cuisine occidentale, et n'inclut pratiquement pas de recettes marocaines authentiques.

c. ChatGPT

Bien qu'il soit très performant, ChatGPT reste un modèle généraliste. Il peut générer des recettes, mais :

— Peut produire des hallucinations (erreurs factuelles ou recettes inventées).

— Ne garantit pas l'authenticité culturelle ni l'exactitude nutritionnelle.

— Ne possède pas de base locale spécifique ou optimisée pour la cuisine marocaine.

1.2 Limites spécifiques observées

— Manque de personnalisation culturelle.

— Peu ou pas de données marocaines dans les bases de données utilisées par ces systèmes.

— Absence de planification de repas contextualisée (par jour, semaine, objectif nutritionnel).

— Hallucinations fréquentes sur les recettes, surtout les spécialités régionales.

— Informations nutritionnelles absentes ou approximatives.

Partie 2 : Conception de notre solution

2.1 Un besoin local et culturel

En tant que marocains, nous ressentons le besoin d'une solution locale, intelligente et culturellement adaptée. La cuisine marocaine est l'une des plus riches et reconnues au monde : elle mérite une représentation fidèle dans les outils numériques modernes. Notre système vise deux objectifs :

— Offrir aux marocains une solution pertinente, fiable, et adaptée à leurs préférences culinaires.

— Partager la richesse de la cuisine marocaine avec le monde entier à travers une IA capable de la comprendre et de la valoriser.

2.2 Présentation de la solution proposée

Notre système repose sur l'architecture RAG (Retrieval-Augmented Generation), combinée à un modèle de langage avancé (LLaMA 4 via l'API Groq). Il intègre une base de données enrichie (recettes internationales + recettes marocaines structurées), un moteur de recherche sémantique basé sur FAISS, et un LLM qui génère des réponses personnalisées, précises et culturellement adaptées.

2.3 Architecture globale du système

Flux général :

— L'utilisateur fournit une requête : liste d'ingrédients, plat recherché, contraintes, etc.

— Le système interroge une base vectorielle (FAISS) construite à partir d'embeddings générés avec all-MiniLM-L6-v2.

— Les recettes pertinentes sont récupérées.

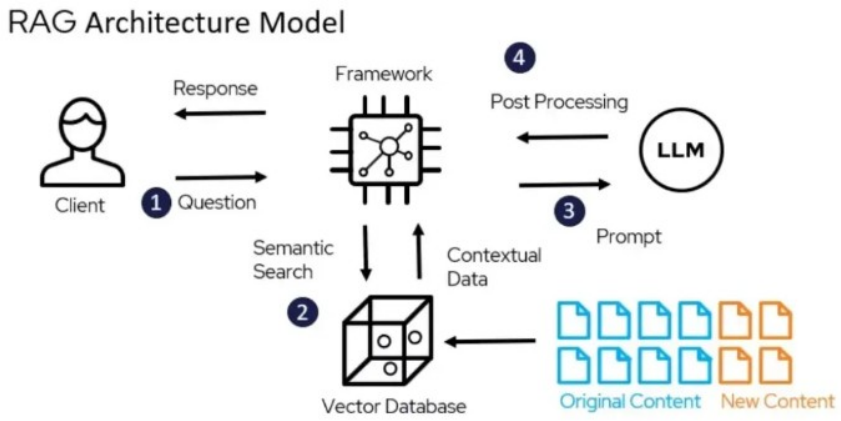


FIGURE 1 – Architecture du système RAG

— Le LLM (LLaMA 4) génère une réponse enrichie, tenant compte du contexte utilisateur, de la recette, et des contraintes alimentaires.

— Une réponse claire, détaillée et contextualisée est affichée.

2.4 Sources de données et nutrition

Nous utilisons deux sources principales :

— Dataset Food.com Recipes and Interactions (Kaggle) : contenant plus de 230 000 recettes avec informations nutritionnelles exactes.

— Recettes marocaines collectées et structurées manuellement, avec nettoyage linguistique et enrichissement sémantique.

L'ensemble des données est vectorisé et indexé localement à l'aide de FAISS, ce qui garantit rapidité et personnalisation dans la recherche.

Tableau comparatif des solutions

TABLE 1 – Tableau comparatif des solutions

Critère	**Supercook**	**Yummly**	**ChatGPT (généraliste)**	**Notre solution (RAG + LLaMA 4)**
Personnalisation des recettes	Faible	Moyenne	Moyenne	Élevée (préférences + contexte)
Cuisine marocaine prise en charge	Non	Non	Partielle	Oui, base dédiée
Qualité des suggestions	Simple appariement	Bonne	Variable (parfois erreurs)	Précise et contextualisée
Informations nutritionnelles	Non	Partielle	Non systématique	Oui, exactes depuis dataset Kaggle
Gestion des contraintes (régime)	Faible	Bonne	Moyenne	Personnalisée dans le prompt
Hallucinations possibles	Faible (recettes fixes)	Faible	Oui	Minimisé par le RAG
Liberté de choisir un plat précis	Non	Partielle	Oui	Oui + contextualisation
Adaptabilité aux préférences	Très faible	Moyenne	Moyenne	Très élevée

Conclusion du chapitre

L'étude des solutions existantes a montré qu'aucune ne répond de manière complète aux besoins spécifiques d'un utilisateur marocain souhaitant à la fois préserver sa culture culinaire et bénéficier des dernières avancées en intelligence artificielle. Grâce à notre approche RAG et à l'intégration d'un LLM puissant (LLaMA 4), notre solution permet une personnalisation intelligente, une prise en charge de la cuisine marocaine, et une précision nutritionnelle rare.

Elle combine le meilleur des deux mondes : la puissance d'un LLM et la pertinence des données locales et vectorisées.

Chapitre 3 : Réalisation

3.1 Vue d'ensemble de l'architecture

L'architecture du projet est divisée en deux grandes parties :

— Une application web full-stack (Frontend + Backend)

— Un système RAG (Retrieval-Augmented Generation) pour générer des réponses basées sur des recettes.

3.2 Pipeline de traitement et système RAG

3.2.1 Structure du projet

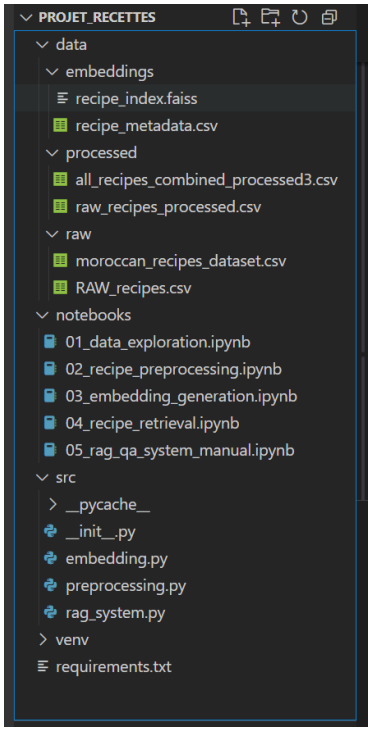


FIGURE 2 – Structure du projet

a) Préparation des données

Nous avons utilisé deux types de sources :

— **Dataset général** : le jeu de données "Food.com Recipes and Interactions" (plus de 230k recettes).

— **Dataset local** : des recettes marocaines collectées manuellement ou à partir de blogs culinaires marocains.

Étapes de préparation :

— Nettoyage et normalisation des champs (ingrédients, instructions, temps de préparation...).

— Unification du format des deux datasets pour créer une structure homogène compatible avec le moteur de recherche.

— Conversion du texte brut en documents exploitables par l'algorithme de RAG.

b) Génération des embeddings

Modèle utilisé : all-MiniLM-L6-v2 pour sa légèreté et sa compatibilité avec FAISS. Problème rencontré : ce modèle est principalement entraîné sur des textes anglais/français standards, donc il ignore ou maltraite des termes en darija comme tajine, kefta, rfissa. Solution : Utilisation d'un filtrage linguistique en amont. Lorsqu'un mot marocain est détecté dans une requête (ex. : tajine), nous générons automatiquement plusieurs variantes : "tajine", "tagine", "moroccan", "north african", "stew". Cela permet au moteur de recherche sémantique d'établir des correspondances plus pertinentes.

c) Indexation et recherche

Utilisation de FAISS pour construire un index vectoriel local. Pour chaque requête utilisateur, les top-k documents les plus proches sont extraits. Ce contexte est ensuite injecté dans une fonction `generate_rag_answer()`, qui construit un prompt enrichi.

d) Modèle de génération

Modèle utilisé : LLaMA 4, accessible via l'API Groq. Avantages :

— Très bonnes performances de génération en français.

— Meilleure gestion du contexte et des nuances linguistiques.

Problème des hallucinations : comme LLaMA est un modèle généraliste, il peut générer des réponses inventées si le contexte est mal défini. Solution : l'usage du RAG permet d'ancrer les réponses dans des documents réels et vérifiés.

3.3 Difficultés rencontrées

TABLE 2 – Difficultés rencontrées et solutions

Problème	**Cause**	**Solution**
Mauvaise reconnaissance des mots en darija	Les modèles d'embedding ne connaissent pas ces termes	Ajout de variantes linguistiques à la requête (tajine → tagine, stew, moroccan)
Incompatibilité des structures de données	Données marocaines hétérogènes	Réécriture des recettes marocaines pour correspondre au format du dataset Food.com
Hallucinations du modèle	LLaMA 4 est généraliste, sans base documentaire propre	Usage du RAG pour insérer un contexte fiable dans le prompt
Intégration difficile du code RAG dans Flask	Le code initial était dans un notebook Jupyter	Refactorisation en modules Python dans `/src/` pour pouvoir les importer dans `app.py`
Pas de modèles d'embedding francophones puissants open-source gratuits	Limite des ressources disponibles en open source	Choix du meilleur compromis (MiniLM) et ajout d'un traitement des requêtes en amont

3.4 Interface utilisateur

Backend (Flask + SQLite)

Endpoints :

— `/register`, `/login`, `/logout`

— `/chat` (intégration RAG)

— `/history`, `/profile`

Base de données :

— Table `users` : stocke les comptes utilisateurs.

```
users (
    id INTEGER PRIMARY KEY,
    username TEXT UNIQUE,
    email TEXT UNIQUE,
    password_hash TEXT,
    created_at TIMESTAMP
)
```

FIGURE 3 – Table users

— Table `messages` : enregistre les conversations liées à chaque utilisateur.

```
messages (
    id INTEGER PRIMARY KEY,
    username TEXT,
    prompt TEXT,
    response TEXT,
    timestamp TIMESTAMP,
    FOREIGN KEY (username) REFERENCES users (username)
)
```

FIGURE 4 – Table messages

Frontend (HTML/CSS/JS)

Pages : `login.html`, `chat.html`, `account.html` **Fonctionnalités** :

— Interface responsive

— Animation de fond et layout moderne

— Historique des chats et affichage de profil

— Envoi des prompts via JavaScript (`fetch API`)

Structure de la partie UI



FIGURE 5 – Structure de la partie UI

Exemple de route d'intégration

```python
@app.route('/chat', methods=['POST'])
def chat():
    if 'username' not in session:
        return jsonify({'error': 'Not authenticated'}), 401

    data = request.json
    prompt = data.get("prompt")
    username = session.get("username")

    if not prompt:
        return jsonify({'error': 'Prompt is required'}), 400

    try:
        answer, source = generate_rag_answer(prompt, top_k=2)
```

FIGURE 6 – Extrait de code d'intégration des routes

Conclusion du chapitre

Ce chapitre a présenté en détail les différentes étapes de la réalisation du projet, de la préparation des données à l'intégration du modèle RAG dans une application web

complète. Nous avons surmonté plusieurs défis techniques, notamment l'hétérogénéité des données, les limites linguistiques des modèles d'embedding, et l'intégration des modules dans un environnement web sécurisé et fonctionnel. Grâce à une architecture bien structurée et une interface utilisateur intuitive, le système final offre une solution innovante, personnalisée et adaptée à la valorisation de la cuisine marocaine. Ces fondations solides ouvrent la voie à de futures améliorations et à une mise en production à plus grande échelle.

Chapitre 4 : Résultats et Interface Utilisateur

4.1 Présentation Générale de l'Interface

Ce projet propose une application web complète, intuitive et responsive, qui permet aux utilisateurs de :

— s'inscrire et se connecter de manière sécurisée,

— interagir avec une IA spécialisée dans la génération de recettes,

— consulter l'historique des conversations,

— gérer leur profil utilisateur.

Chaque fonctionnalité a été testée pour garantir une expérience fluide.

4.2 Page de Connexion

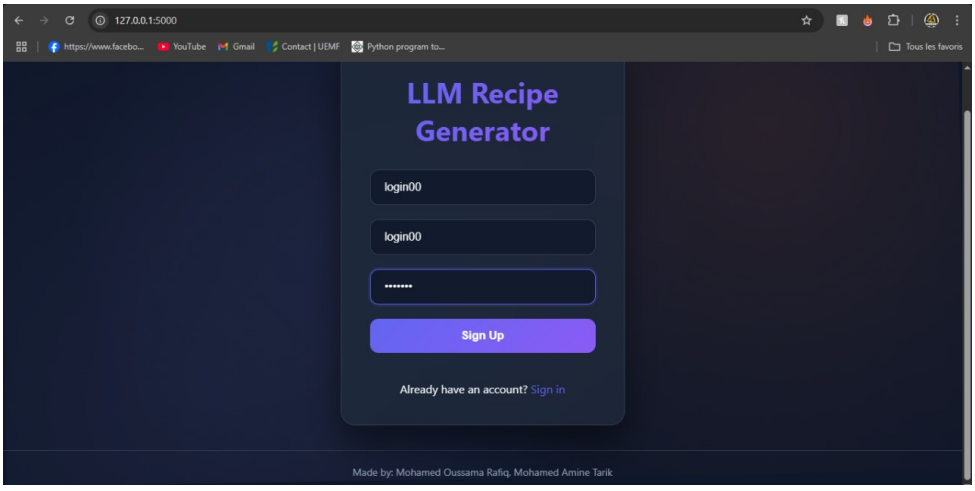


FIGURE 7 – Page de Connexion

La page de connexion permet aux utilisateurs de se connecter ou de créer un compte. Elle intègre :

— un design responsive,

— une interface animée fluide,

— une validation côté client,

— une gestion sécurisée des sessions.

4.3 Page de Chat IA

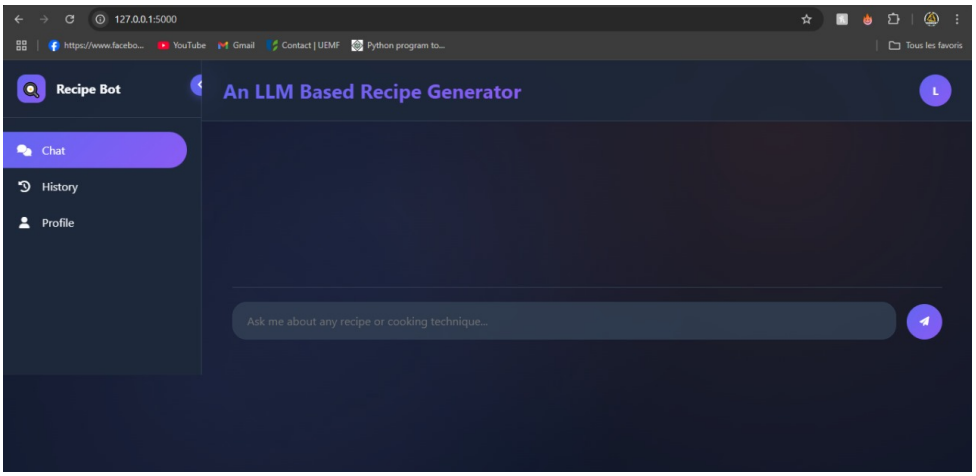


FIGURE 8 – Page de Chat IA

C'est l'espace principal d'interaction avec le système RAG intégré à LLaMA 4 via l'API Groq. L'utilisateur peut :

— poser des questions ou proposer des ingrédients,

— recevoir des suggestions de recettes,

— voir les sources utilisées (grâce au RAG),

— profiter d'une interface moderne avec effets de chargement et auto-resize.

4.4 Exemple de Test

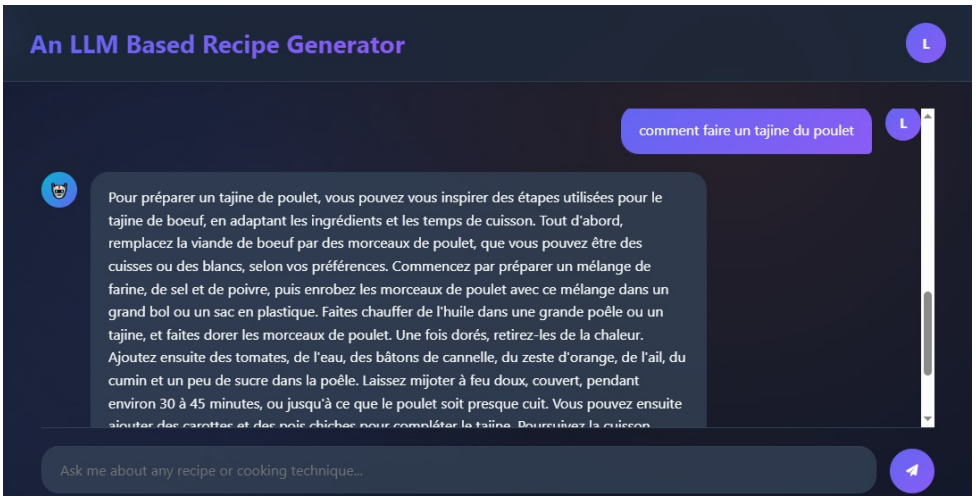


FIGURE 9 – Exemple de Test : Plat Marocain (Tajine)

Cet exemple montre la réponse du système à une question sur un plat marocain. Grâce à notre stratégie de reformulation (ajout de termes comme “tajine”, “tagine”, “moroccan”, “north africa”), l’IA comprend mieux la requête et fournit une réponse pertinente.

4.5 Historique des Conversations

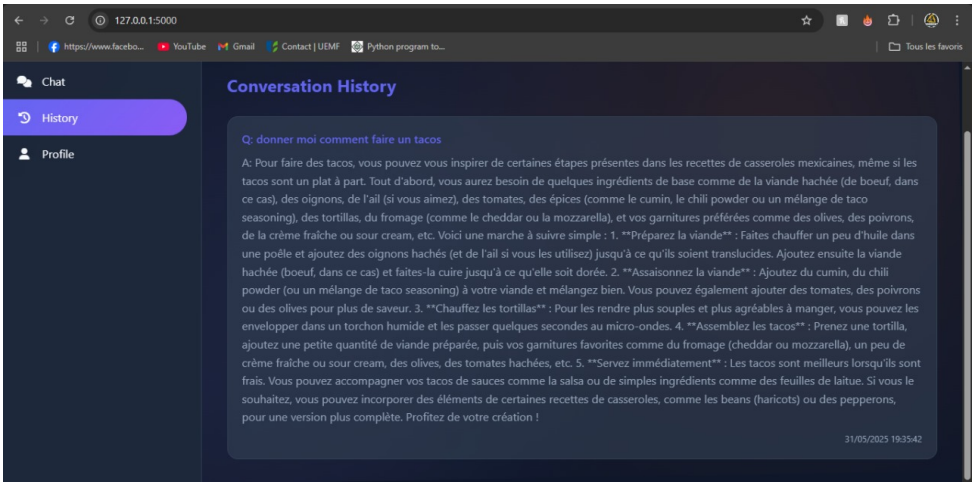


FIGURE 10 – Historique des Conversations

L'utilisateur peut consulter l'historique de ses interactions avec l'IA, filtrer ou relire les suggestions précédentes. Ceci permet un usage prolongé du système sans perte d'information.

4.6 Profil Utilisateur

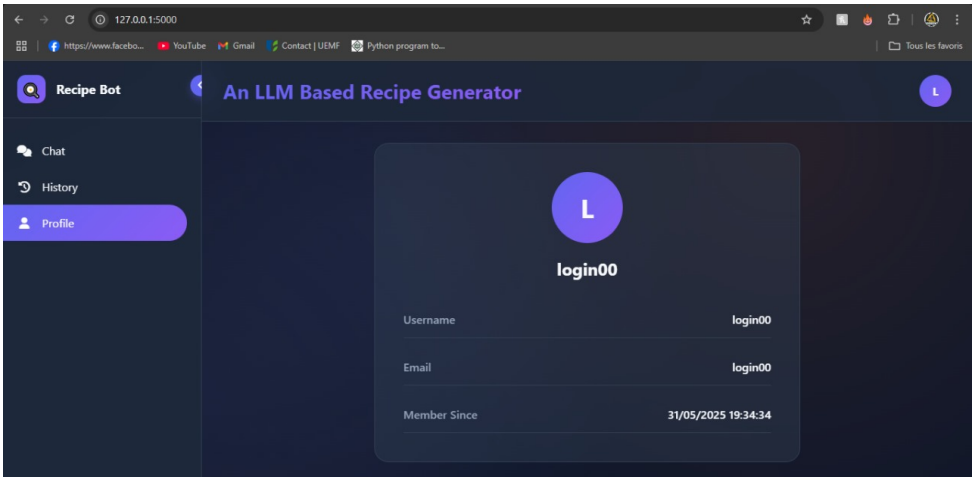


FIGURE 11 – Profil Utilisateur

La section profil permet de visualiser les informations personnelles de l'utilisateur, de suivre la date d'inscription et d'assurer une meilleure personnalisation.

Conclusion Générale

Ce projet a consisté à concevoir et développer un générateur de recettes et planificateur de repas intelligent, basé sur un système de récupération augmentée par génération (RAG) couplé à un grand modèle de langage (LLaMA 4 via API Groq). L'objectif principal était de proposer une solution locale valorisant la richesse de la cuisine marocaine tout en offrant la flexibilité de personnaliser les recettes selon les préférences des utilisateurs.

Nous avons d'abord analysé les limites des solutions existantes, notamment l'absence de ressources marocaines adaptées et les limites des modèles généralistes comme ChatGPT qui peuvent générer des hallucinations. La mise en place d'un système RAG a permis de pallier ces limitations en combinant une base documentaire spécialisée et un modèle LLM puissant.

La conception du pipeline a impliqué le nettoyage et la structuration des données, la génération d'intégrations textuelles adaptées, ainsi que la construction d'un index de recherche efficace via FAISS. Des difficultés spécifiques liées au traitement de la darija et des termes culinaires marocains ont été identifiées et contournées grâce à une approche hybride et à la reformulation intelligente des requêtes.

Le développement d'une application web complète, sécurisée et ergonomique a permis d'offrir une interface accessible pour tous, intégrant authentification, historique des conversations et gestion du profil. Les tests réalisés confirment la pertinence et la robustesse du système.

En conclusion, ce travail illustre la pertinence des technologies RAG et LLM pour créer des applications culinaires innovantes et adaptées aux contextes locaux. Pour aller plus loin, il serait intéressant d'enrichir la base documentaire marocaine, d'optimiser les

modèles d'intégration pour la darija, et d'intégrer des fonctionnalités avancées comme la génération automatique de plans nutritionnels personnalisés.

Bibliographie

[1] Kaggle. (2021). *Food.com Recipes and Interactions Dataset.* Disponible sur : `https://www.kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions` (Note : Ce dataset inclut RAW_recipes.csv utilisé dans le projet.)

[2] Kaggle. (2023). *Moroccan Recipes Dataset.* Disponible sur : `https://www.kaggle.com/datasets/idrisskh/moroccan-recipes-dataset` (Note : Veuillez vérifier si l'URL correspond exactement au dataset marocain que vous avez utilisé si vous l'avez obtenu d'un autre utilisateur Kaggle précédemment mentionné.)

[3] Facebook AI Research. (2023). *FAISS : A library for efficient similarity search.* Disponible sur : `https://faiss.ai/`
Référence académique associée : Johnson, J., Douze, M., & Jégou, H. (2017). *Billion-scale similarity search with GPUs.* arXiv preprint arXiv :1702.08734.

[4] Groq, Inc. (2024). *Groq Cloud Documentation.* Disponible sur : `https://console.groq.com/docs/overview` (Modèle LLM spécifique utilisé : meta-llama/llama-4-scout-17b-16e-instruct)

[5] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357-362.

[6] McKinney, W. (2010). Data Structures for Statistical Computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (Vol. 445, pp. 51-56).

[7] Reimers, N., & Gurevych, I. (2019). Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3988–3996. (Modèle d'embedding spécifique utilisé : all-MiniLM-L6-v2)

[8] Cloudflare. (2024). *Qu'est-ce que les embeddings ?*. Disponible sur : `https://www.cloudflare.com/fr-fr/learning/ai/what-are-embeddings/`

[9] Google Developers. (2024). *Machine Learning Crash Course : Embeddings*. Disponible sur : `https://developers.google.com/machine-learning/crash-course/embeddings?hl=fr`

[10] Lewis, P., Oguz, B., Yarats, D., Singh, N., Goff, H., Piktus, F., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459-9474.

[11] Wikipédia. (2025). *Modèles de langage de grande taille (LLM)*. Disponible sur : `https://fr.wikipedia.org/wiki/Mod%C3%A8le_de_langage_de_grande_taille`

[12] YouTube. (2024). *Introduction au RAG (Retrieval-Augmented Generation)* [Vidéo]. Disponible sur : `https://youtu.be/sVcwVQRHIc8?si=dEkOcvZAb5V8EVry`